



7.2. Searching in Arrays

Previous Sections:

 [7. Arrays](#)

 [7.1. Operations on Arrays](#)

Contents:

[1. Linear Search](#)

[Implementation using the Array Class](#)

[2. Binary Search](#)

[Implementation using the Array Class](#)

[Summary](#)

[References](#)

One of the primary reasons we store data inside arrays is so that we can find it later.

Imagine a database containing thousands of student records, a music application storing millions of songs, or an online store managing a large inventory of products. Storing the data is only part of the challenge—the real question is how quickly we can locate a particular item when needed.

Searching algorithms are designed to solve this problem. Given a target value (often called the **key**), a searching algorithm determines whether the value exists in the array and, if so, returns its location.

In this section, we will study two of the most important searching algorithms:

- **Linear Search**
- **Binary Search**

Although both algorithms solve the same problem, they use very different approaches and have different performance characteristics.

1. Linear Search

Linear Search is the simplest searching algorithm. The algorithm starts from the first element of the array and compares each element with the target value one by one until the target value is found, or the end of the array is reached.

Because the algorithm examines elements sequentially, it works on both sorted and unsorted arrays.

Consider the following array:

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 10 | 50 | 30 | 70 | 80 | 20 | 90 | 40 | 45 | 77 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
  0     1     2     3     4     5     6     7     8     9
```

Suppose we want to find: **Key = 30**

The algorithm performs the following comparisons:

- Compare 10 with 30 → Not found
- Compare 50 with 30 → Not found
- Compare 30 with 30 → Found

The search stops immediately when the element is found.

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 10 | 50 | 30 | 70 | 80 | 20 | 90 | 40 | 45 | 77 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
  ↑
Compare with 30
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 10 | 50 | 30 | 70 | 80 | 20 | 90 | 40 | 45 | 77 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
  ↑
Compare with 30
```

```

+---+---+---+---+---+---+---+---+---+---+
| 10 | 50 | 30 | 70 | 80 | 20 | 90 | 40 | 45 | 77 |
+---+---+---+---+---+---+---+---+---+---+
          ↑
        Found

```

- Advantages of Linear Search: Very easy to implement. Works on unsorted arrays. No preprocessing required
- Disadvantages of Linear Search: Can be slow for large datasets. May need to examine every element
- Time Complexity for the worst case: **$O(n)$** because every element may need to be checked.

Implementation using the Array Class

For this section, we only need: `__init__()` ; `traverse()` ; `insert_end()` ; and a new searching method.

```

class Array:

    def __init__(self, maximum):
        self.maximum = maximum
        self.n = 0
        self.arr = [0] * maximum

    def traverse(self):
        print(self.arr[:self.n])

    def insert_end(self, item):

        if self.n < self.maximum:
            self.arr[self.n] = item
            self.n += 1

    def linear_search(self, key):

        for i in range(self.n):

```

```

        if self.arr[i] == key:
            return i

    return -1

```

```

arr = Array(10)

arr.insert_end(10)
arr.insert_end(50)
arr.insert_end(30)
arr.insert_end(70)
arr.insert_end(80)
arr.traverse()

print(arr.linear_search(30))

```

[10, 50, 30, 70, 80]

2 → The value 30 is found at index 2.

2. Binary Search

Binary Search is a significantly faster searching algorithm. Unlike Linear Search, Binary Search does not examine every element one by one. Instead, it repeatedly divides the search space into smaller portions until the target element is found. This strategy is known as **Divide and Conquer**.

However, Binary Search has one important requirement: ***The array must already be sorted.***

Without a sorted array, Binary Search cannot determine which half of the data should be discarded.

Consider the following sorted array:

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Suppose we want to find: **Key = 42**

- Step 1 - Set: **Low = 0; High = 14**

Calculate the middle position:

$$Mid = \left\lfloor \frac{Low + High}{2} \right\rfloor$$

Mid = 7 → Element at index 7 is 29. Since: **42 > 29**, the target must be on the right side. Discard the left half.

- Step 2 - New search range:

```
+-----+
| 31 | 36 | 42 | 47 | 53 | 55 | 62 |
+-----+
```

Middle element: 47. Since: **42 < 47**, the target must be on the left side. Discard the right half.

- Step 3 - New search range:

```
+-----+
| 31 | 36 | 42 |
+-----+
```

Middle element: 36 → Since: **42 > 36**, so we discard the left half.

- Step 4 - Remaining element:

```
+-----+
| 42 |
+-----+
```

The value has been found.

The process: **29 → 47 → 36 → 42**. Only four comparisons were needed.

Why Binary Search is Faster

Suppose the array contains 15 elements. Linear Search may require 15 comparisons. Binary Search only requires 4 comparisons. As the array grows larger, this difference becomes even more significant. For example:

Number of Elements	Linear Search	Binary Search
100	100	7
1,000	1,000	10
1,000,000	1,000,000	20

This efficiency is why Binary Search is one of the most important algorithms in computer science.

- Advantages of Binary Search: Extremely fast for large datasets. Uses Divide and Conquer. Requires far fewer comparisons
- Disadvantages of Binary Search: Requires a sorted array. More complex than Linear Search. Not suitable for unsorted data
- Time Complexity for the best case is $O(1)$, and for the worst case is $O(\log \times n)$

Implementation using the Array Class

For this section, we only need: `__init__()` ; `traverse()` ; `insert_end()` ; and a Binary Search method.

```
class Array:

    def __init__(self, maximum):
        self.maximum = maximum
        self.n = 0
        self.arr = [0] * maximum

    def insert_end(self, item):
        if self.n < self.maximum:
            self.arr[self.n] = item
            self.n += 1
```

```
def binary_search(self, key):
    low = 0
    high = self.n - 1

    while low <= high:
        mid = (low + high) // 2

        if self.arr[mid] == key:
            return mid
        elif key > self.arr[mid]:
            low = mid + 1
        else:
            high = mid - 1

    return -1
```

```
arr = Array(15)

values = [3, 6, 8, 12, 14, 17,
          25, 29, 31, 36, 42, 47, 53, 55, 62]

for value in values:
    arr.insert_end(value)

print(arr.binary_search(42))
```

10 → The value 42 is located at index 10.

Summary

In this section, we studied two fundamental searching algorithms used with arrays: **Linear Search** and **Binary Search**.

Linear Search works by examining elements one by one from the beginning of the array until the target value is found. Although it is simple to implement and works on unsorted arrays, its worst-case time complexity is **O(n)**, making it less efficient for large datasets.

Binary Search improves performance by applying the Divide and Conquer technique. By repeatedly dividing a sorted array into smaller sections, it can locate an element in **$O(\log n)$** time. This makes Binary Search significantly faster than Linear Search for large collections of data, provided that the array is already sorted.

These two algorithms demonstrate how the organization of data can greatly affect performance. Understanding searching techniques is an important step toward building efficient programs and prepares us for more advanced algorithms and data structures in later chapters.

References

https://drive.google.com/file/d/18wXJmAyHVrloU2XspveSV1hGezBzSWEC/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/linear-search/>

<https://www.geeksforgeeks.org/dsa/binary-search/>

<https://www.youtube.com/watch?v=C2apEw9pgtw>

Next Section:

 [8. Linked Lists](#)